Extracting Data with Web Scraping

Explore how to extract data from web pages using web scraping techniques with Python. Learn to send HTTP requests, parse HTML content with BeautifulSoup, extract specific elements like titles and links, navigate webpage structures, and export data for analysis. This lesson provides practical skills to gather online data when APIs are not available.

Introduction to web scraping

Web scraping is a method for extracting data from web pages. With web scraping, we can extract data in HTML, XML, or JSON format from webpages, parse it, and extract the relevant data. We can create scripts to automatically retrieve and parse data from web pages according to a schedule and extract online data, such as comments from a forum or a social media platform or the latest price of product items from Amazon.

Web scraping can also be used as a one time process to extract relevant data. It has a wide range of applications, including data mining, data analysis, online market research, and more. It's a useful tool for extracting data from websites that do not provide an API.

Python is an awesome tool for web scraping. It has two great libraries for web scraping called `requests` and `Beautifulsoup`.

The `requests` library

Ask

The `requests` library lets us send HTTP requests to websites and handle the response. The most common type of request for our purpose is a `get` request. We

use a `get` request to retrieve information from a server or a service. If the request is successful, the server will return a response, which is the data we requested, usually in HTML/JSON format.

The `Beautifulsoup` library

After creating a successful request and fetching the response using the `requests` library, we can use the `Beatifulsoup` library to parse and navigate the returned content.

Response codes

After sending a `get` request to a server, a response code will be returned. These codes communicate the status of the HTTP request. The response codes are grouped into classes and codes of the same range.

Response codes between 200 and 299 indicate success, codes between 300 and 399 indicate redirection, codes between 400 and 499 indicate a client-side error, and codes between 500 and 599 indicate a **server-side error**.

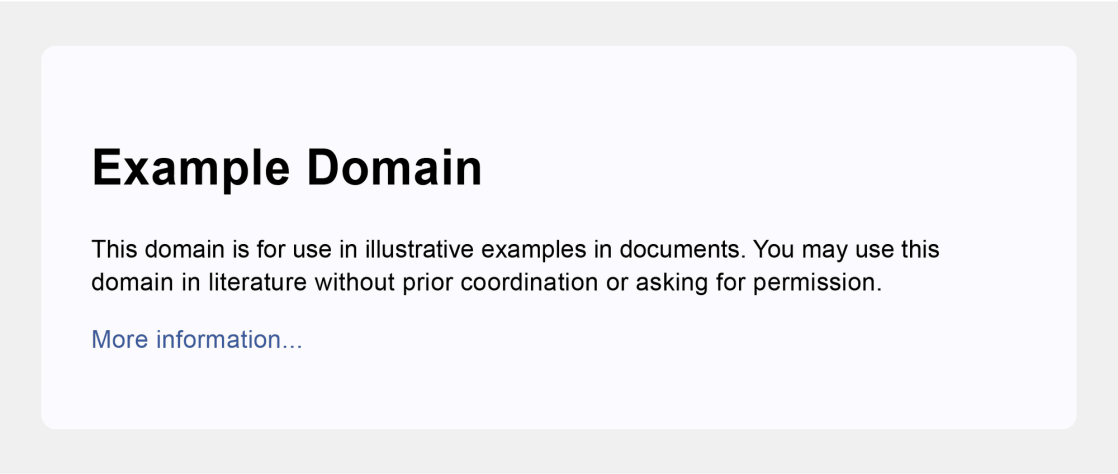
Some common responses are:

- **200 OK**: The request was successful, and the server has returned the requested response.
- **302 Found**: The requested resource has been temporarily moved to a new location.
- **400 Bad Request**: The request was invalid or malformed.
- **404 Not Found**: The requested resource could not be found on the server.
- **500 Internal Server Error**: An unexpected error occurred on the server while processing the request.

Ask

Tutorial

Let's go over a quick tutorial on web scraping using Python. We'll web scrape the following example website:



Example website to web scrape

Here's the HTML content of this website (we'll scrape the `title` and the `body` elements from this website):

index.html HTML

```html
<!DOCTYPE html>
<html>
 <head>
  <title>
   Example Domain
  </title>
  <meta charset="utf-8"/>
  <meta content="text/html; charset=utf-8" http-equiv="Content-type"/>
  <meta content="width=device-width, initial-scale=1" name="viewport"/>
  <style type="text/css">
   body {
        background-color: #f0f0f2;
        margin: 0;
```

HTML content

Sending a `get` request

The first step in web scraping is to send a `get` request to the webpage.

The following script will send a `get` request to `https://www.example.com` using the `requests` library, then check if the request is successful by verifying that the response code is equal to `200`, and if so, print a message accordingly.

main.py Python 3.10.4

```
import requests

URL = "https://www.example.com"

# Send a GET request to "https://www.example.com"
response = requests.get(URL)

# Grab the response status code
response_code = response.status_code

# Check status code and print a message accordingly
if response_code == 200:
    print('The HTTP request is successful \n')
```

Run

Sending a get request and getting the response code

- **Line 6:** We send a `get` request to `https://www.example.com`.
- **Line 9:** We grab the response code.
- **Lines 12–15:** We check if the response code is `200`. Print a message accordingly.

Parsing the HTML content

Ask

Next, we create a `BeautifulSoup` object called `soup` by using its built-in parser called `html.parser` to parse the HTML content from the response and print the

HTML content.

main.py Python 3.10.4

```
# Parsing the response using BeautifulSoup
soup = BeautifulSoup(response.text, 'html.parser')

# Print the HTML response
print(soup.prettify())
```

Run

Parsing the response and creating a Beautifulsoup object

- **Line 2:** We parse the HTML response into a BeautifulSoup object called `soup`.
- **Line 5:** We prettify and print the HTML content.

`BeautifulSoup` has multiple types of parsers, some are more suitable for web pages written in XML and others for HTML. `html.parser` is a lightweight and fast parser for HTML web pages with good performance when the structure of the HTML is well-formed.

Extracting data

After creating `soup`, we can use various methods to navigate and extract the specific elements from the HTML content. Let's check out some examples:

Ask

Extracting the title element

We can use `soup.title` to extract the title element from the HTML response. Then add the method `.string` to extract the contents.

main.py Python 3.10.4

```
# Parsing the response using BeautifulSoup
soup = BeautifulSoup(response.text, 'html.parser')

# Extract title
title = soup.title.string
print(title)
```

Run

Extracting the title of the web page

Extracting links

We can use the method `soup.find_all()` to create a list of all elements of a specific type. For example, we can create a list of all the hyperlinks from the webpage using `soup.find_all(name = 'a')`.

main.py Python 3.10.4

```
# Extract links
links = soup.find_all(name='a')
print(f'Links: {links}')
```

Ask

Run

Extracting the links of the web page

Then we can iterate over the hyperlinks in the list and extract additional information. We can use the `.get()` method for each hyperlink to extract specific elements or `.contents` to grab the content. For example, we can run the following script to extract the element `href` from each link, which indicates the destination of the hyperlink, and the title of the link.

main.py Python 3.10.4

```
# Extract the href and title of each link
for link in soup.find_all(name='a'):

    href = link.get(key='href')
    link_title = link.contents[0]

    print(f'\nLink Destination: {href}')
    print(f'Link Title: {link_title}')
```

Run

Extracting the links of the web page

Navigating the webpage

Other than extracting elements and content from the HTML response, we use `Beatifulsoup` to navigate the webpage. This is useful when we know exactly where the content on the webpage is located.

Ask

Let's try to extract the main text of the webpage. The main text is a paragraph attribute (p) and can be found in the `body` element inside a `div` attribute.

index.html HTML

```html
<body>
  <div>
   <h1>
    Example Domain
   </h1>
   <p>
    This domain is for use in illustrative examples in documents. You may
    domain in literature without prior coordination or asking for permiss
   </p>
   <p>
    <a href="https://www.iana.org/domains/example">
     More information...
```

Body element of https://www.example.com/

Extracting the body element

We can use the method `soup.select()` to extract the `body` element from the webpage.

main.py Python 3.10.4

```python
# Extract the body
body = soup.select('body')[0]
print(body.prettify())
```

Ask

Run

Extracting the body element

Navigating the `body` element and extracting the `p` element

After extracting the `body` element, we can use the following methods to navigate within it:

- `body.find_next()`: To find the next child element
- `body.find_all_next()`: To find all the next child elements
- `body.find_all_previous()`: To find all previous parent elements
- `body.find_all_sibling()`: To find all sibling elements
- `body.findChild()`: To find all children elements of a specific type of element
- `body.children`: To create an iterable to iterate over all children elements
- `body.parents`: To create a generator object to iterate over the parent elements

For example, the following script extracts the `body` element's child attribute `div`. Then, it extracts the contents of the next paragraph within the `div` attribute. Finally, it prints the paragraph's content.

main.py Python 3.10.4

```
# Extract the next child element - div
div = body.find_next()

# Find the next paragraph in the div element
p = div.find_next('p').contents[0]

# Print the paragraphs contents
print(p)
```

Ask

Run

Extracting the p (paragraph) element‘s content

- **Line 2:** We extract the child element from the body.
- **Line 5:** We navigate to the nearest paragraph, and extract the contents.

Exporting the data

Once we’ve extracted the data from the webpage, we can save it to a file or database for further analysis. For example, we can save all the extracted content in a text file for later usage.

main.py Python 3.10.4

```
# Write the extracted data into example_contents.txt
print('Writing the data into a text file...')
with open('example_contents.txt', 'w' ) as write_file:

    write_file.write(f'Title: {title}')
    write_file.write('\n' + f'Hyper Link:{str(links)}')
    write_file.write('\n' + f'Paragraph: {p}')

# Read the contents of example_contents.txt
print('\nReading the data\n')
with open('example_contents.txt', 'r') as read_file:

    print(read_file.read())
```

Run

Exporting the data to a text file

Ask

- **Line 3:** We use a context manager to create a writable empty file called `example_contents.txt`.

- **Lines 5–8**: We append to the file the data we found earlier (e.g., link title, link

Ask